Code blocks appear throughout Ruby applications. Often they're used in conjunction with iterators—methods that return successive elements from some kind of collection, such as an array:

```
animals = %w[ ant bee cat dog elk ]    # create an array
animals.each {|animal| puts animal }   # iterate over the contents
```

Each integer *N* implements a `times` method, which invokes an associated block *N* times:

```
3.times { print "Ho! " }    #=>  Ho! Ho! Ho!
```

The `&` prefix operator allows a method to capture a passed block as a named parameter:

```
def wrap &b
  print "Santa says: "
  3.times(&b)
  print "\n"
end
wrap { print "Ho! " }
```

Within a block, or a method, control is sequential except when an exception occurs.

Exceptions

Exceptions are objects of the `Exception` class or its subclasses. The `raise` method causes an exception to be raised. This interrupts the normal flow through the code. Instead, Ruby searches back through the call stack for code that says it can handle this exception.

Both methods and blocks of code wrapped between `begin` and `end` keywords intercept certain classes of exceptions using `rescue` clauses:

```
begin
  content = load_blog_data(file_name)
rescue BlogDataNotFound
```